


**TECHNICAL UNIVERSITY OF MOLDOVA**
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

# LABORATORY REPORT

## OPERATING SYSTEMS LABORATORY 1

### The OS as a Resource Manager

**Done by:** st. gr. FAF-241, Lucian-Adrian Gavril

**Checked by:** univ. assist., Reițman Patricia

**Chișinău, 2026**

# TABLE OF CONTENTS

ABSTRACT ..... 3

INTRODUCTION ..... 4

1 ENVIRONMENT AND SYSTEM IDENTITY ..... 5

1.1 Host Identification ..... 5

1.2 Command-to-Observation Mapping ..... 5

2 FILES AND ACCESS PERMISSIONS ..... 6

2.1 Root Directory Hierarchy ..... 6

2.2 File Creation, Modification, and Permission Enforcement ..... 6

2.3 Answers to Observe Prompts (Files) ..... 6

3 PROCESSES AND THE /PROC VIRTUAL FILE SYSTEM ..... 7

3.1 Process Hierarchy and Process 1 ..... 7

3.2 Process State Inspection ..... 7

3.3 Answers to Observe Prompts (Processes) ..... 8

4 MEMORY MANAGEMENT ..... 9

4.1 System Memory Accounting ..... 9

4.2 Per-Process Resident Set Size ..... 9

5 DEVICES AND FILE SYSTEM MOUNTING ..... 10

5.1 Block Storage and Mount Points ..... 10

5.2 Device Special Files ..... 10

CONCLUSIONS ..... 11

## ABSTRACT

This laboratory report investigates the four fundamental responsibilities of an operating system: file system management, process management, memory management, and device/storage management. The investigation was conducted on an Ubuntu 24.04.4 LTS virtual machine running Linux kernel 6.8.0-136-generic on x86_64 architecture. Using standard POSIX utilities and the Linux `/proc` virtual file system, live system states were measured, inspected, and verified. The report documents permissions enforcement, process lifecycle monitoring, physical memory accounting, and block device abstraction.



**Keywords:** operating systems, linux, resource management, processes, virtual filesystem, memory management

# INTRODUCTION

The operating system acts as an intermediary resource manager standing between application programs and the physical hardware. Rather than permitting user applications direct, uncoordinated access to physical storage, CPU registers, and RAM, the OS kernel abstracts hardware components into uniform interfaces and coordinates their allocation across concurrent workloads.

The objective of Laboratory 1 is to observe and verify the four primary resource management roles of the Linux kernel from the command-line interface:

a) **File System Management**: Hierarchical namespace organization, directory traversal, file metadata, and Discretionary Access Control (DAC) permission enforcement;

b) **Process Management**: Process creation, process hierarchy anchored at PID 1 (`systemd`), process state inspection via `/proc`, and signal termination;

c) **Memory Management**: System-wide physical memory and swap accounting via `free` and `/proc/meminfo`, and per-process resident set sizing (`VmRSS`);

d) **Device and Storage Management**: Block device mapping via `lsblk`, virtual device nodes under `/dev`, and filesystem mounting verified via `df -h` and `mount`.

# 1 ENVIRONMENT AND SYSTEM IDENTITY

## 1.1 Host Identification

The commands were executed in a dedicated Ubuntu 24.04.4 LTS virtual machine under user account `ubuntu`:

```
$ whoami
ubuntu

$ uname -a
Linux ubuntu 6.8.0-136-generic #136-Ubuntu SMP PREEMPT_DYNAMIC x86_64 GNU/Linux

$ uptime
14:24:47 up 21 min, 0 user, load average: 0.00, 0.00, 0.00

$ cat /etc/os-release | grep PRETTY_NAME
PRETTY_NAME="Ubuntu 24.04.4 LTS (Noble Numbat)"
```

## 1.2 Command-to-Observation Mapping

Table 1.1 outlines the experimental operations and the operating system concepts they demonstrate.

**Table 1.1 – Laboratory 1 command-to-observation guide**

| Resource | Commands | OS Mechanism Demonstrated |
|---|---|---|
| Identity | `whoami`, `uname -a`, `uptime` | Execution context, kernel release, and system uptime. |
| Files | `ls -la /`, `ls -l`, `chmod 600` | VFS namespace, inode metadata, and DAC permission check. |
| Processes | `ps aux`, `ps -p 1`, `/proc/PID/status` | Process table, PID 1 hierarchy, and scheduler wait state. |
| Memory | `free -h`, `/proc/meminfo`, `VmRSS` | Host physical RAM accounting vs. process resident set size. |
| Devices | `df -h`, `lsblk`, `ls -l /dev` | Device special files, block storage, and VFS mount points. |

# 2 FILES AND ACCESS PERMISSIONS

## 2.1 Root Directory Hierarchy

Inspecting the root directory / with `ls -la /` reveals essential system directories:

- `/etc`: System-wide configuration files and lookup tables (e.g., `passwd`, `hosts`);
- `/home`: User workspace directories containing individual user profiles;
- `/usr`: System binaries, shared libraries, headers, and documentation;
- `/dev`: Special device node files representing physical and pseudo devices.

## 2.2 File Creation, Modification, and Permission Enforcement

A working file `note.txt` was created, inspected, and modified:

```
$ echo "hello operating systems" > note.txt
$ ls -l note.txt
-rw-r--r-- 1 ubuntu ubuntu 24 Sep 28 14:24 note.txt

$ chmod 600 note.txt
$ ls -l note.txt
-rw------- 1 ubuntu ubuntu 24 Sep 28 14:24 note.txt
```

## 2.3 Answers to Observe Prompts (Files)

a) **Ownership and Group**: The file owner is `ubuntu`; the owning group is `ubuntu`. In Linux, every file inode associates an owning UID and GID to evaluate access permissions;

b) **Mode Representation after `chmod 600`**: The mode displays as `-rw-------`. The leading hyphen denotes a regular file. The first triplet (`rw-`) grants read and write authority to the owner. The second (`---`) and third (`---`) triplets completely deny read, write, and execute permissions to group members and other users;

c) **Root Directories**: Two observed directories directly under / are `/etc` and `/home`.

# 3 PROCESSES AND THE /PROC VIRTUAL FILE SYSTEM

## 3.1 Process Hierarchy and Process 1

The process tree is anchored at process identifier 1:

```
$ ps -p 1 -o pid,ppid,comm,args
  PID  PPID COMMAND         COMMAND
    1     0 systemd         /sbin/init

$ ps aux | wc -l
103
```

PID 1 is `systemd` (running `/sbin/init`), the initial user-space process spawned directly by the kernel after hardware initialization. The process listing reported 103 lines, corresponding to 1 header line and 102 active processes.

## 3.2 Process State Inspection

A background process was launched and inspected via the `/proc` pseudo-filesystem:

```
$ sleep 300 &
[1] 1784

$ ps -ef | grep "[s]leep"
ubuntu 1784 1750 0 14:24 ? 00:00:00 sleep 300

$ grep -E '^(Name|State|Pid|VmRSS):' /proc/1784/status
Name:   sleep
State:  S (sleeping)
Pid:    1784
VmRSS:  1964 kB

$ kill 1784
[1]+  Terminated              sleep 300
```

### 3.3 Answers to Observe Prompts (Processes)

a) **Process 1 Identity**: PID 1 is `systemd` (`/sbin/init`). It acts as the ancestor of all user-space processes and reaps orphaned child processes;

b) **Active Process Count**: The system snapshot revealed approximately 102 active processes;

c) **Sleeping Process State**: The state is `S (sleeping)`, representing an interruptible sleep state. The process is removed from the CPU runqueue while waiting on a timer event and consumes zero CPU cycles.

# 4 MEMORY MANAGEMENT

## 4.1 System Memory Accounting

Physical and virtual memory parameters were captured using `free -h` and `/proc/meminfo`:

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       407Mi       3.0Gi       976Ki       614Mi       3.4Gi
Swap:             0B          0B          0B
```

The VM has 3.8 GiB of total physical memory, with 3.0 GiB completely unallocated (`free`) and 3.4 GiB immediately usable (`available`), which includes memory that can be reclaimed from disk caches. Swap space was unconfigured (0 B).

## 4.2 Per-Process Resident Set Size

A test process resident set size was measured from `/proc/PID/status`:

```
$ grep VmRSS /proc/1797/status
VmRSS: 1968 kB
```

Even when idle in sleep state, the process occupies 1,968 kB of physical RAM to hold executable code segments, runtime libraries, environment variables, and the user stack.

# 5 DEVICES AND FILE SYSTEM MOUNTING

## 5.1 Block Storage and Mount Points

The physical and virtual storage topology was inspected with `df -h` and `lsblk`:

```
$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        19G  1.9G   17G  11% /

$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   20G  0 disk
+-sda1   8:1    0   19G  0 part /
sr0     11:0    1 1024M  0 rom
```

The root filesystem is hosted on partition `/dev/sda1` formatted as ext4, occupying 1.9 GB of 19 GB total space (11% utilization). The block device is backed by virtual disk `sda` (20 GB).

## 5.2 Device Special Files

In Linux, devices are exposed in the filesystem through special character and block device nodes under `/dev`:

```
$ ls -l /dev/cdrom
lrwxrwxrwx 1 root root 3 Sep 28 14:04 /dev/cdrom -> sr0

$ ls -l /dev/sr0
brw-rw----+ 1 root cdrom 11, 0 Sep 28 14:04 /dev/sr0
```

The symbolic link `/dev/cdrom` references block device `/dev/sr0` (major 11, minor 0). This confirms that the OS abstracts hardware interfaces into the standard file namespace.

## CONCLUSIONS

Laboratory 1 demonstrated the four foundational roles of an operating system kernel through direct command-line inspection and system introspection. The experimental observations verified that:

a) The file system manager structures raw disk space into a hierarchical directory namespace and enforces permissions based on metadata stored in inodes;
b) The process manager schedules execution threads, maintains the process hierarchy starting at PID 1, and exposes real-time state via `/proc`;
c) The memory manager tracks physical RAM and page allocation, distinguishing unallocated memory from reclaimable cache;
d) The device manager abstracts heterogeneous physical and virtual hardware into standard device nodes under `/dev`.